

HTML:

```
<!DOCTYPE html>
<html>
<head><title>Title</title>
<link rel="stylesheet" type="text/css" href="style.css">
</head>
```

```
<body>
<div><span><p><a href="url">Link</a>
<li>Item</li></ul>
<ol>
<form action="/submit" method="post">
<input type="text" name="username" placeholder="Enter">
<input type="password" name="password" value="Submit">
<button></button>
</form>
src="path", href="url"
```

```
<table border="1">
<tr>
<th>Heading 1</th>
<th colspan="2">St Details</th>
</tr>
<tr>
<td rowspan="2">Mark</td>
<td>ID</td>
<td>Grade</td>
</tr>
<tr>
<td>ID</td>
<td>Grade</td>
```

CSS

```
element { } * → universal
.class { }
#id { }
elem, class { }
```

margin: Outside
padding: Inside
content → padding → border → margin

display: block/inline/inline-block/
flex/grid/none
position: static/relative/absol/
fixed/sticky

Flex

flex-direction:
justify-content (Horiz Align)
align-items (vert Align)
gap { flex-wrap

Grid

grid-template-columns: 200px 1fr 200px
grid-rows: 100px auto 80px;
gap 10px;
header {
grid-column: 1/4 (spans mul col)
}

@media screen & (max-width: 700px)
{ //CSS }

Combinators

A B → Descendant (All B's inside A)
A > B → Child (All B's direct child of A)
A + B → Adjacent sibling (B immediately after A)
A ~ B → General sibling (All B's after A)

Pseudo Classes

hover, :nth-child(N), :nth-child(even/odd)

JavaScript

Variables:
let x=5; (Block)
const y=2;
var z=3; (Func)
dtypes:
String, Number, Boolean, Array, Object, null

JS DOM

Select:
get Element By Id('id')
querySelector('.class')
querySelectorAll('div') → Ret nodeList
get Elements By ClassName('class1')
(One can work for class2)
→ Ret array of elements

Modify DOM

el.value
el.textContent
el.innerHTML
el.setAttribute('toggle', 'on')
el.addEventListener('submit', func(e))
e.preventDefault();
type → click, change, keypress, load

document.createElement('div')
var newEl =
document.body.appendChild(newEl)
removeChild(Elem)

Functions

```
function name(x, y){  
  return x+y;  
}  
const fn = function(x, y){ return x+y; }  
const fn = () => { }  
fn = x => x+2;
```

Loop

arr.forEach(item => { });

Array Methods

map()
filter()
reduce()
find()
push(), pop()
slice(), splice()

JSON

```
const jsonStr = { "name": "John", "cars":  
  ["c1", "c2"] };  
const obj = JSON.parse(jsonStr);  
obj.name, obj.cars[0]  
obj["name"]  
const newObj = { name: "Tom", age: 25 };  
const newJson = JSON.stringify(newObj);  
<script src="script.js"></script>
```

JQuery

\$(Selector).action() → Auto runs
\$(document).ready(function(){ });
// Short → \$(function){ };

Selectors

\$("#div") // all div elem
\$("#myId")
\$(".myClass")
\$("#p#id") // para with id
\$("p>a") // Direct child of p
\$("a:first") // first anchor
\$("p:odd")
\$("button")
\$("checkbox")

CSS Modify

.addClass(" ")
.toggleClass()
.css("color", "blue")
.html() // gets html content
.html("New"); // new html content
.text()
.text("Adds text");

.append("text")
.prepend("text")
.after(), .before() // Inserts content outside selected
.remove(); // Deletes node & child

Events

\$("#a").click(function(){ });
Other: blur, focus, hover, keydown, load, submit, select, mousemove

Effects

.hide("slow"), .show("fast"), .toggle(100)
.fadeOut(), .fadeIn(), .slideUp(), .slideDown()
\$("#box").animate({
 // CSS;
 3, 2000, function(){ }
});

AJAX

GET Req:

```
$.get("script.php", function(data, status){  
  //  
});
```

POST Req:

```
$.post("script.php", { name: "Ali", age: 43 },  
function(data, status){  
  console.log(status);  
});
```

Full AJAX:

```
$.ajax({  
  url: "/api/tasks",  
  type: "GET",  
  data: { id: 123 },  
  success: function(result){ }  
  error: function(xhr, status, error){ }  
});
```

API Requests

XML Http Req

```
const xhr = new XMLHttpRequest();  
xhr.open('GET', url);  
xhr.send(); → on right bottom
```

Fetch APIs

```
fetch(url)
  .then(res => res.json())
  .then(data => { _ })
  .catch(err => { _ });
```

GET-Req:

```

fetch(url)
  .then(r => r.json())
  .then(d => console.log(d))

```

POST Rec.:

```
fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type':
    'application/json'
  },
  body: JSON.stringify(data)
})
```

Premises

```
const myPromise = new Promise(
  (resolve, reject) => {
    // async op
    if (success)
      resolve(data); // can be string
    else
      reject(data);
```

Page No. State:

Promise states:
 Pending: Initial state
 Fulfilled: Success
 Rejected: Failure

Using Promise: (.then
Chain)

```

my Promise
    .then(result => {
        return process(result);
    })
    .then(data => { })
    .catch(error => { console.error(error); })
    .finally(() => { // always executes? })

```

```
Promise.all([p1, p2, p3])
  .then(results => {
    // all resolved
  });
```

Waits for all promises
Fails if any rejects

```
Promise.race([P1, P2])
  , then (result => { // first to
    settle });
  Returns 1st settled
```

Promise.allSettled([...])
 .then(res => {
 // waits for all promises to settle

```
Promise.any([p1, p2])
  .then(res => {
    // at least one successful resolve.
```

Async/Await

```

async function getData(){
  try{
    const res = await
      fetch(url);
    const data = await
      res.json();
    return data;
  } catch(err){
    console.error(err);
  }
}

```

Async Patterns

Sea Exec:

```
async function seq() {
  const a = await task1();
  const b = await task2();
  return [a, b];
}
```

Parallel Experi

```
async function parallel() {
  const [a, b] = await
    Promise.all([
      task1(), task2()
    ]);
  return [a, b];
}
```

From 12/1/2010

```

Error handling:
try {
    await asyncFn();
} catch (e) { }
// With .catch()
asyncFn().catch(e) { }

```

Conduct: EOTCHAS

Common: 50/100

```
// Seq:
for (let i of items)
{ await process(i); }
// Parallel
await Promise.all(
  items.map(i =>
    process(i))
);
```

Ergebnisse:

forget to await:
// Wrong
`const data = fetch(url);`

Correct
`const data = await
fetch(url);`

Assume Euro Rate

Async func Ret.
 async function fn() {
 return val; }
 Always returns promise
 f().then(x → { }).

Cookies

```
// Read all cookies
const allCookies = document.cookie;

// Parse cookies
function getCookie(name) {
    const cookies = document.cookie
        .split('; ');
    for (let c of cookies) {
        const [key, val] = c.split('=');
        if (key == val) return val;
    }
    return null;
}
```

```
// Set cookie
document.cookie = "username=Moiz"

// With expiry
document.cookie = "username=Moiz;
    expiry=Thu, 18 Dec 2025 12:00:00 UTC;
    path=/"
```

// With options
doc.cookie = "theme = dark; max-age = 3600;"

// Delete cookies (set expiry in past
->"

Vanilla Ice Cream

```
const timerId = setTimeout(() => {
  console.log("Delayed 2 seconds, 1000ms → 1s");
}, 2000);
```

```
// Now this executes repeatedly pausing by the delay
const interval = setInterval(() => {
  console.log("Tick - -");
}, 1000);
```

Stop them.

```
clearTimeout(timerId); } clearInterval(intervalId);  
setInterval(() => { const now = new Date();  
doc.getElementById('clock').innerHTML += `<b>Time: <b>`  
`${now.toLocaleTimeString()}`; }, 1000);
```

APT Reg*

```

xhr.onload = function() {
  if (xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    printToScreen('Success, name = ' + (data.name));
  } else {
    // 
  }
}

```